

Atlas / Argus - Implementation Plan Update

FRONTEND CODEBASE UPDATE - 2026-05-29

This update supersedes implementation-plan rows that say the frontend is entirely missing. The uploaded web app ZIP contains a real apps/web React/Vite scaffold. Treat the original plan as correct for architecture, security, and backend gaps, but update the frontend work from "create the app" to "finish and integrate the app."

Corrected current-codebase summary

Aspect	Updated reality
Frontend	apps/web exists: React + Vite + TypeScript + Tailwind + TanStack Query + React Router + Lucide.
Routes	Login, case list, dashboard, claims, conflicts, timeline, provenance, reports, audit.
API client	Central fetch wrapper uses VITE_API_BASE_URL, credentials: include, JSON case conversion, typed error handling, and 409 detection.
Still missing	services/bff, apps/marketing, production report export UI, document upload/ingestion UI, source management UI, global search, and full tests.
Build verification	Not verified in this container because dependencies are not installed. Verify with npm ci && npm run build in the project environment.

Updated rule for frontend work

- **Keep apps/web.** Do not throw away the scaffold; harden it and connect it to the BFF and FastAPI surface.
- **Add services/bff.** Human login/session concerns still belong outside the Atlas core schema. The BFF owns sessions and server-side API keys.
- **Add apps/marketing.** The public site remains separate from the logged-in SPA.
- **Prefer adapters over rewrites.** Where the current frontend expects /api/v1/auth/*, either preserve that prefix in the BFF or update all callers consistently.

Revised MVP implementation tasks

Order	Task	Notes
1	Wire/harden BFF/session skeleton	Skeleton exists in atlas-bff-docs-reports.zip; wire login/me/logout, HttpOnly cookies, CSRF, server-side API keys, throttling, and production DB/session storage.
2	Normalize app API routes	Current app expects /api/v1/auth/login, /api/v1/auth/me, /api/v1/auth/logout. Choose and standardize the BFF prefix.
3	Wire case data through BFF	Proxy/camelCase public events, accident detail, provenance, conflicts, timeline, and audit. Cache event_id <-> slug.
4	Fix conflict claim candidates	Return full candidates with claimId, value, source, reliability tier, timestamp, claim type, winning/superseded flags. Submit stable claimId, not an index.
5	Complete 409 conflict-modified UX	Use returned conflict, currentVersion, latestActivity, and accidentRecord; refresh the panel and let the reviewer retry.
6	Extract shared EvidencePanel	One component for source header, values, metadata, provenance chain, linked conflicts, claim history, and audit hash.
7	Complete Reports flow	Routes/Reports.tsx exists; finish preview, immutable PDF generation, version history, storage, auth, and download wiring.
8	Complete document upload/ingestion UI	Routes/Documents.tsx exists; finish PDF upload, parse status, extracted text, failures, ingest action, and source linkage.
9	Build apps/marketing	Home, Platform, Use Cases, How It Works, Security, Compliance, About, Request Access, Contact, legal pages.
10	Add tests and CI	Typecheck, lint, Playwright for login/cases/conflicts/409/reports/upload.

Conflict queue implementation correction

The original implementation plan correctly treats the conflict queue as the killer screen. The uploaded frontend already has a conflict route and mutation hooks, but it also reveals a concrete issue that must be fixed before production use.

Do not map selected claim index to `conflict.claimIds[selectedClaimIdx]`. The displayed evidence claims may not be in the same order as `claimIds`. The API/BFF should return full claim candidates and the UI should submit the exact selected `claimId`.

Required candidate type

```
interface ConflictClaimCandidate {  
  claimId: string;  
  fieldName: string;  
  fieldValue: unknown;  
  sourceId: string;  
  sourceName: string;  
  sourceReliabilityTier: number;  
  createdAt: string;  
  claimType: ClaimType;  
  isWinning: boolean;  
  isSuperseded: boolean;  
}
```

Updated conflict data flow

- BFF loads conflict detail and relevant provenance/evidence claims.
- BFF returns `ConflictDetailResponse` with conflict, candidates, and history.
- `ResolveForm` uses a radio value of `candidate.claimId`.
- `Resolve` submit body sends `expected_version` and exactly one of `winning_claim_id` or `manual_override_value`, plus required rationale.
- On HTTP 409, the panel updates to the returned current version and shows the latest activity inline, not just a toast.

Frontend test targets

- Login redirects to `/app` and `/auth/me` populates role and permissions.
- Analyst role sees conflict actions disabled; reviewer/admin can resolve and reopen.
- `Resolve` picks the correct `claimId` even when candidates are sorted differently from raw `claimIds`.
- 409 modified-conflict response refreshes the panel and allows retry with the new expected version.
- Reports page shows preview, generates a version, and lists/downloads prior exports.
- Document upload shows success, parse failure, and ingest-in-progress states.

Atlas / Argus — Implementation Plan

From blueprint to buildable. Every recommendation is mapped to the real code in the ZIP.

0. Corrections to the blueprint after reading the actual code

The blueprint was directionally right, but four concrete facts from the source change the implementation. Read these first — they ripple through auth, the conflict UI, and every import path.

- 1. **The Python package is `atlas`, not `atlas_argus`.** Your prompt's example paths (`src/atlas_argus/...`) are wrong. The real import root is `src/atlas/`, **console script** `atlas = "atlas.presentation.cli.commands:app"` . All new backend files go under `src/atlas/` .
- 2. **All API routers are mounted under `/api/v1` (see `app.py::_include_routers`).** So the real paths are `/api/v1/conflicts`, `/api/v1/accidents/{event_id}`, `/api/v1/accidents/{event_id}/provenance`, `/api/v1/ingestion/sources/{source_id}`, `/api/v1/public/events/...`, `/api/v1/admin/...` .
- 3. **The role model is `analyst` / `reviewer` / `admin`** (DB CHECK-constrained in `ApiKeyModel`), plus an optional tenant binding (`tenant_role` ∈ `OWNER` / `MEMBER` / `READ_ONLY`). The blueprint's "viewer/curator/admin/tenant-admin" naming is wrong — use the real enum (`atlas.domain.enums.Role`). Map: **analyst = read-only**, **reviewer = analyst + resolve/reopen conflicts + ingest**, **admin = reviewer + destructive ops** (merge, rebuild, reproject, outbox).
- 4. **There is no human-user, password, or session model anywhere.** Authentication is *purely* API-key: `X-API-Key` → `ApiKeyModel(user_id: UUID, role, tenant_id, tenant_role)`. `user_id` is a bare UUID stamped into `ClaimHistory` /activity logs — not a foreign key to any `users` table. **This is the single biggest implication for the build:** the browser cannot hold an API key, and there is no login. So the BFF must (a) own a *new* human-identity + session layer, and (b) bridge each logged-in human to a server-side API key. The good news: because the existing RBAC and tenant isolation are entirely keyed off `ApiKeyModel`, you can reuse all of it with **zero backend authorization changes** by provisioning one scoped API key per human user and keeping it server-side.

Two more load-bearing details the UI must respect:

- **Conflict resolution uses optimistic concurrency.** `resolve` / `reopen` take an `expected_version`; a stale version raises `ConflictModifiedError` → **HTTP 409** with a rich body (`current_version`, `latest_activity`, `refreshed_conflict`, `refreshed_accident_record`). The conflict queue UI *must* handle 409 as a first-class "someone changed this; here's the new state — review and retry" flow, not a generic error.
- **Resolve requires exactly one of** `winning_claim_id` **or** `manual_override_value` (enforced by a Pydantic `model_validator`). The form must encode this as a mutually-exclusive choice.

1. Current codebase summary

Aspect	Reality
Type	Backend plus frontend scaffold. apps/web exists (React/Vite/TypeScript/Tailwind/TanStack Query/React Router); apps/marketing and production hardening remain.
Framework	FastAPI ≥0.121, SQLAlchemy async, Pydantic v2, Uvicorn/Gunicorn, Typer CLI.
Layout	Clean architecture: <code>domain/</code> (pure), <code>application/</code> (use cases + DTOs + UoW), <code>infrastructure/</code> (db, ingestion, event bus, observability), <code>presentation/</code> (api routers + schemas + CLI).
APIs	~20 routers under <code>/api/v1</code> : accidents, conflicts, provenance, ingestion, public, search, nl_search, maps, chronos, orion, hermes, argus, causality, cms, editorial, audit, metering, tenancy, admin.
Domain modules	<code>causality</code> (HFACS/SHELO), <code>crossref</code> , <code>maps</code> , <code>metering</code> , <code>nl_search</code> , <code>publication</code> , <code>search</code> , <code>tenancy</code> , <code>services</code> (16 files incl. ingestion, winner policy), <code>interfaces</code> .
DB / migrations	PostgreSQL/PostGIS via Alembic. Append-only claims/conflicts/projections + outbox. Multi-DB topology: <code>TENANT_DATABASE_URL</code> (NOBYPASSRLS, RLS-enforced), <code>SYSTEM_DATABASE_URL</code> , optional <code>PUBLIC_DATABASE_URL</code> . ORM in <code>infrastructure/db/orm_models.py</code> .
Auth/security	<code>X-API-Key</code> → <code>ApiKeyModel</code> (HMAC-hashed via <code>security.py</code> , in-process LRU cache). Roles <code>analyst</code> / <code>reviewer</code> / <code>admin</code> . <code>require_role()</code> dependency. Rate limit, CORS, HSTS configurable in <code>config.py</code> . No human login/session.
Ingestion	Generic use case <code>IngestSourceData</code> behind <code>POST /ingestion/sources/{source_id}</code> (<code>raw_payload</code> + <code>claims[]</code> + <code>captured_at</code> + idempotency). NTSB eADMS importer (<code>application/ingestion/sources/ntsb_eadms.py</code> pure mapper + <code>infrastructure/ingestion/ntsb_eadms_reader.py</code> + <code>presentation/cli/ntsb.py</code>) feeds that same use case.

Tests	tests/ mirrors the source tree; unit (no DB) vs integration (DB) split via Makefile (<code>make test-unit / test-integration</code>); pytest-asyncio, coverage on.
Deploy	Dockerfile , docker-compose.yml , deploy/free/ (Caddy + Postgres + Redis + PgBouncer + API + outbox worker), gunicorn_conf.py , start.sh , ops/ (k6 load test, runbooks).
Frontend scaffold (updated)	apps/web exists: React/Vite/TypeScript/Tailwind/TanStack Query/React Router with login, cases, claims, conflicts, timeline, provenance, reports, and audit routes. Still missing apps/marketing and production hardening.
Missing backend for MVP	(1) human-user + session + RBAC bridge (BFF); (2) unstructured-document (PDF/docket) ingestion adapter; (3) report-export module → PDF; (4) request-access lead capture; (5) uploaded-file + report storage models.

2. MVP decision

Strict scope. Build exactly these; defer everything else.

Feature	Why it matters	Existing backend support	Missing work	Priority
Public marketing site	Generate qualified leads; pass first-glance credibility	None	Full static site	P0
Request-access flow	Capture leads (no signup)	None	leads table + 1 endpoint + form	P0
Login + session-safe app shell	Humans can't hold API keys	API-key RBAC exists; no human auth	Users + sessions + BFF	P0 (foundational)
Case switcher	Navigate cases (events)	GET /public/events	BFF passthrough + UI	P0
Case dashboard	Orient on a case	GET /accidents/{id} , /admin/metrics	UI	P0
Claims view	See reconciled record + per-field sources	projection (/accidents/{id}) + provenance	UI	P0
Conflict queue	The core decision workflow	GET /conflicts?event_id=	UI + 409 handling	P0 (centerpiece)
Resolve / reopen flow	Make + record decisions	`POST /conflicts/{id}/resolve	reopen`	UI form, version handling
Provenance / evidence panel	Defend any fact	GET /accidents/{id}/provenance	UI	P0
Timeline view	Reconstruct sequence	GET /public/events/{slug}/timeline , /chronos/...	UI	P0
Basic audit visibility	Show integrity	GET /audit/... , conflict /history	Read-only UI	P0 (thin)
PDF/docket upload + ingestion	Get real evidence in	IngestSourceData use case	New adapter + upload route + storage	P0
Defensible PDF report export	The paid deliverable	projection + provenance + history reads	New export module + render + storage	P0

Explicitly deferred (do not build for MVP): causality/HFACS UI, Orion entity graph, maps UI, NL-search UI, tenant onboarding UI, editorial workflow UI, SSO/SAML, webhooks, client portal, cross-accident search. All have backing endpoints already; they're V1+.

3. Recommended frontend architecture

Stack decision: React + Vite + TypeScript + Tailwind + shadcn/ui + Lucide + TanStack Query + React Router. Reasoning:

- **Vite + React Router over Next.js for the app.** The logged-in app is a pure SPA talking to a JSON API; you do not need SSR/RSC, and Next's server model would muddy the BFF boundary. Vite keeps the app a clean client that only ever talks to *your BFF*, never FastAPI directly.
- **Next.js (or Astro) only for the marketing site**, as a separate deployable — it benefits from SSG/SEO and has zero auth. Keep it in its own

folder; do not entangle it with the app.

- **TanStack Query** is non-negotiable here: the entire app is server-state (cases, claims, conflicts, provenance). It gives you caching, request dedup, optimistic updates for resolve, and clean invalidation after a resolve/reopen.
- **shadcn/ui + Tailwind + Lucide** matches your existing prototype and gives accessible primitives (Dialog, Drawer, Table, Toast) you'd otherwise hand-roll — important for the conflict drawer and confirmations.

```
apps/  
  marketing/           # Next.js or Astro — public site, no auth  
  web/                 # Vite + React SPA — the logged-in app  
    src/  
      app/             # app bootstrap: providers, router, query client, error boundary  
      routes/          # route components mapped 1:1 to URLs (thin; compose features)  
      features/        # domain-organized logic + UI, the heart of the app  
        cases/         # case list, switcher, dashboard  
        claims/        # claims table + claim detail  
        conflicts/     # conflict queue, drawer, resolve/reopen forms ← invest most here  
        provenance/    # evidence panel, chain tree  
        timeline/      # chronological view  
        documents/     # upload + ingestion status  
        reports/       # preview + generate + download  
        audit/         # read-only activity views  
      components/     # shared, domain-agnostic UI (Table, StatusChip, Drawer, EmptyState)  
      lib/             # api client (calls BFF), query keys, formatters, constants  
      hooks/           # cross-feature hooks (useCurrentUser, useCase, usePermissions)  
      types/           # TypeScript types mirroring backend DTOs (see $7)  
      styles/          # tailwind config tokens, globals  
    services/  
      bff/             # Node (Fastify/Express) or FastAPI — session + API-key bridge
```

Folder intent: `routes/` stay thin and only wire URL → feature; **all real logic lives in** `features/<domain>/` (each owns its components, hooks, api calls, types). `components/` is strictly reusable and domain-free. `lib/api` only ever points at the BFF origin — it never knows FastAPI exists.

4. Browser-safe authentication plan

Why no raw X-API-Key in the browser: the key is a bearer credential that maps directly to a role and tenant in `ApiKeyModel`. Anything in browser JS is readable by the user, any XSS, and any extension. A leaked key = full API access at that role. The existing API is *designed* for trusted server-to-server callers.

Approach: a thin BFF with server-side sessions that bridges to per-user API keys. This reuses 100% of the existing RBAC/tenant model.

Concrete flow:

1. **User logs in at the BFF** (`POST /auth/login`, email + password for MVP). BFF verifies against a new `users` table (Argon2/bcrypt hash).
2. **BFF creates a session** — a random opaque id stored server-side (`sessions` table or Redis, which you already run), mapped to `user_id`.
3. **Browser stores a cookie** — `HttpOnly`, `Secure`, `SameSite=Lax`, `__Host-` prefix, no JS access. Cookie holds *only* the session id.
4. **BFF calls FastAPI using a server-side API key provisioned for that user:** on user creation, the BFF mints an `ApiKeyModel` (via the existing `atlas bootstrap/key` path) carrying that user's `user_id` + `role` (+ tenant binding), stores the *raw* key encrypted in the BFF's `users` row (or a secrets store), and attaches it as `X-API-Key` on every upstream call. The key never leaves the server.
5. **FastAPI enforces tenant/RBAC unchanged** — because the key already encodes role + tenant, `require_role()` and RLS work exactly as today. The `user_id` correctly lands in `ClaimHistory` when that user resolves a conflict.
6. **Frontend receives safe JSON only** — the BFF strips upstream headers, never forwards the key, and shapes errors (notably mapping FastAPI 401/403/409 into clean app responses).

Roles in the UI: read `role` from `GET /auth/me` (BFF-provided). Gate write actions (resolve/reopen/ingest) to `reviewer / admin`; hide/disable for `analyst`. This is a UX nicety — the *real* enforcement stays server-side in FastAPI.

MVP auth = email/password + server session + per-user key bridge. Defer to later: SSO/SAML/OIDC (enterprise will ask, but it's a V1+/sales-driven add; the BFF session boundary is exactly where you'll later plug an IdP without touching FastAPI), MFA, SCIM provisioning.

5. Public website implementation

Separate `apps/marketing` deployable (Next.js/Astro). No auth, no app dependencies.

Page	Route	Sections	Components	CTA	Data needed
Home	/	Hero, problem, solution, feature overview, use cases, workflow, security strip, preview, CTA, footer	HeroSection , ProblemSection , SolutionSection , FeatureCards , UseCaseCards , WorkflowSteps , EvidenceChainSection , SecurityStrip , PreviewImage , CtaSection , Footer	Request access	Static
Platform	/platform	Claims model, conflict resolution, provenance, timeline	FeatureDeepDive , DiagramBlock	Request access	Static
Use cases	/use-cases	Litigation, safety investigation, airline safety	UseCaseDetail x3	Request demo	Static
How it works	/how-it-works	Ingest→reconcile→resolve→export + philosophy	WorkflowSteps , PhilosophyCallout	See workflow	Static
Security	/security	Encryption, RLS isolation, audit hashes, key handling, hosting	SecurityFeatureList	Contact	Static
Compliance	/compliance	Retention, residency, processing roles, sub-processors	ComplianceTable	Contact	Static
About	/about	Team, domain credentials, why	TeamGrid	—	Static
Request access	/request-access	Qualifying form	RequestAccessForm	Submit	POST /api/leads (BFF)
Contact	/contact	Form + security/legal contacts	ContactForm	Submit	POST /api/contact (BFF)
Privacy / Terms / AUP	/legal/*	Legal copy	LegalDoc	—	Static (counsel-reviewed)

Component plan (shared): HeroSection , ProblemSection , EvidenceChainSection , ConflictResolutionSection , SecurityStrip , UseCaseCards , WorkflowSteps , RequestAccessForm , Footer .

Homepage copy is already finalized in the blueprint (§5 of the PDF) — use it verbatim: headline *“The record you can defend.”* and the sober, no-“AI”-in-hero positioning. Don’t re-invent it.

6. Logged-in app UX implementation

Route	Purpose	Layout	Components	API (via BFF → FastAPI)	MVP
/app	Landing / case picker	Sidebar + content	CaseSwitcher , RecentActivity	GET /public/events	P0
/app/cases	All cases	Sidebar + table	CaseTable	GET /public/events	P0
/app/cases/:caseId	Case dashboard	Sidebar + metric cards	CaseHeader , MetricCards , ConflictSummaryWidget	GET /accidents/{id} , /admin/metrics	P0
/app/cases/:caseId/claims	Reconciled record	3-column	ClaimsTable , ClaimDetail , EvidencePanel	projection + provenance	P0
/app/cases/:caseId/conflicts	Conflict queue	3-column + drawer	ConflictTable , ConflictDetail , ResolveForm , EvidencePanel	GET /conflicts , `POST .../resolve	reopen , /history`
/app/cases/:caseId/timeline	Sequence	List/timeline + right panel	TimelineList , TimelineItem , EvidencePanel	/public/events/{slug}/timeline	P0
	Full				

/app/cases/:caseId/provenance	evidence chain	Tree + detail	ProvenanceTree , ClaimHistoryList	/accidents/{id}/provenance	P0
/app/cases/:caseId/reports	Export	Preview + actions	ReportPreview , GenerateButton , ReportList	report routes (new)	P0
/app/sources	Sources + upload	Table + upload	SourceTable , DocumentUpload , IngestionStatus	/hermes/sources , documents (new)	P0
/app/audit	Activity log	Table	AuditTable	/audit/...	P0 (thin)
/app/settings/team	Roles/keys	Table	MemberTable	/admin , /tenancy	V1

Three-column working layout (the spine of the app):

- **Left (list rail):** the working set for the current screen — claims list, or conflict queue, or timeline items. Sortable/filterable table. Selecting a row drives the center.
- **Center (focus):** the selected item — a claim’s current value + field, or a conflict’s competing claims + resolve form, or a timeline entry. This is where action happens.
- **Right (evidence):** the **shared** EvidencePanel — source, original value, reliability tier, ingestion timestamp, provenance chain, linked conflicts, claim history, audit hash. It reacts to whatever is selected in the center. Built once, reused on claims / conflicts / timeline.

Implement the 3-column as a single CaseWorkspaceLayout with resizable panes; each route fills the three slots with its own list/center components and the shared EvidencePanel .

7. Conflict queue implementation (the killer screen)

Component hierarchy

```
ConflictQueuePage
├─ ConflictFilters      (status, field_name, source, free text)
├─ ConflictTable        (left rail: rows = conflicts)
│   └─ ConflictRow      (field_name · StatusChip · #claims · updated_at)
├─ ConflictDetailPanel  (center)
│   └─ ConflictHeader    (field, status, version)
│   └─ CompetingClaimsList
│       └─ ClaimCandidateCard (value · source · reliability tier · timestamp · select radio)
│   └─ ResolveForm       (radio: pick winner XOR manual override; required rationale)
│   └─ ReopenForm        (only when RESOLVED; required rationale)
│   └─ ConflictHistoryPanel (activity log, keyset-paginated)
└─ EvidencePanel        (right: provenance for the selected claim)
```

State management

- **TanStack Query** for all reads: ['conflicts', caseId, filters] , ['conflict', id] , ['conflictHistory', id] , ['provenance', caseId] .
- **Mutations** for resolve/reopen with **optimistic update** of the conflict row + projection, and onError rollback. On success, invalidate ['conflicts', caseId] , ['conflict', id] , ['conflictHistory', id] , and ['accident', caseId] (projection changed).
- Track expected_version from the loaded conflict; send it on submit.

API calls

- **List:** GET /api/v1/conflicts?event_id={caseId}&limit=&offset=
- **Detail:** GET /api/v1/conflicts/{id}
- **History:** GET /api/v1/conflicts/{id}/history?limit=50&cursor=
- **Resolve:** POST /api/v1/conflicts/{id}/resolve body { expected_version, winning_claim_id? , manual_override_value?, reason }
- **Reopen:** POST /api/v1/conflicts/{id}/reopen body { expected_version, reason }

Form validation

- Exactly one of winning_claim_id / manual_override_value (mirror the backend model_validator); disable submit otherwise.

- `reason` required for court-defensibility (backend allows empty; *you* should require it in the UI — it's the whole point).
- Reopen only enabled when `status === 'RESOLVED'`.

Error handling — the critical one: on 409 (`ConflictModifiedError`), do *not* show a toast and bail. Parse the 409 body, show an inline banner (“This conflict changed since you opened it — reviewer X did Y”), refresh the panel with the returned `conflict + accident_record + latest_activity`, `update expected_version`, and let the user re-decide. This turns a race condition into a safe, on-the-record interaction.

Empty / loading / permission states

- Empty: “No open conflicts for this case.” (good news — say so plainly).
- Loading: skeleton rows in the table, skeleton in the panel.
- Permission: `analyst` sees everything read-only; resolve/reopen controls are disabled with a tooltip (“Requires reviewer role”). Real enforcement is server-side (403).

Keyboard shortcuts (this audience works fast): `j/k` move selection, `1-9` select competing claim, `r` focus rationale, `⌘/Ctrl+Enter` submit resolve, `o` reopen, `e` toggle evidence panel.

TypeScript types (mirror the real Pydantic DTOs):

```
type Role = 'analyst' | 'reviewer' | 'admin';
type ConflictStatus = 'OPEN' | 'RESOLVED';
type ClaimType = 'RAW' | 'CONFIRMED' | 'MANUAL_OVERRIDE' | 'SUPERSEDED';

interface ClaimSummary {
  id: string; eventId: string; fieldName: string; fieldValue: unknown | null;
  sourceId: string; rawSnapshotId: string | null; claimType: ClaimType;
  createdAt: string; createdBy: string | null; supersededByClaimId: string | null;
}
interface ConflictSummary {
  id: string; eventId: string; fieldName: string; status: ConflictStatus;
  version: number; lastModifiedReason: string; lastModifiedNote: string | null;
  winningClaimId: string | null; resolvedAt: string | null; resolvedBy: string | null;
  createdAt: string; updatedAt: string; claimIds: string[];
}
interface ProjectionFields {
  eventId: string; projectionVersion: number; fields: Record<string, unknown>;
  completenessScore: number; unresolvedConflictFields: string[]; updatedAt: string;
}
interface ActivityLogEntry {
  id: string; conflictId: string; sequence: number;
  fromStatus: ConflictStatus | null; toStatus: ConflictStatus;
  modifierType: 'USER' | 'INGESTION' | 'SYSTEM'; modifierId: string | null;
  reason: string; versionAtMoment: number;
  claimsSnapshot: Record<string, unknown> | null; createdAt: string;
}
interface ResolveRequest {
  expectedVersion: number; winningClaimId?: string;
  manualOverrideValue?: unknown; reason: string;
}
interface ResolveResponse { conflict: ConflictSummary; accidentRecord: ProjectionFields; }
interface ConflictModified409 {
  detail: string; modifierReason: string | null; currentVersion: number;
  latestActivity: ActivityLogEntry | null; conflict: ConflictSummary; accidentRecord: ProjectionFields;
}
```

(The BFF should camelCase these; FastAPI emits snake_case.)

8. Evidence / provenance panel implementation

A single shared `EvidencePanel` consumed by claims, conflicts, and timeline.

UI layout (top→bottom): Source header (title, type/kind, reliability tier chip) → Values block (original raw value vs extracted/normalized claim) → Metadata (ingestion timestamp, `source_record_id`, claim type) → Provenance chain (claim → snapshot → ingestion run → source) → Linked conflicts (chips → jump to conflict) → Claim history (created/superseded/confirmed) → Audit hash (mono, copyable, with verify affordance).

Components: `EvidencePanel`, `SourceHeader`, `ReliabilityTierChip`, `ValueDiff`, `ProvenanceChain`, `LinkedConflicts`, `ClaimHistoryList`, `AuditHashRow`.

API: `GET /api/v1/accidents/{eventId}/provenance` — returns paginated sections (`claims`, `claim_histories`, `conflicts`, `conflict_activity`,

`projection_history`) with per-section cursors (`claims_cursor` , `claim_history_cursor` , ...). For a selected claim, filter the provenance payload client-side to that `claimId` / `fieldName` ; paginate sections lazily.

Types: reuse `ClaimSummary` , `ConflictSummary` , `ActivityLogEntry` ; add:

```
interface ProvenanceResponse {
  eventId: string;
  claims: ClaimSummary[];
  claimHistories: ActivityLogEntry[];
  conflicts: ConflictSummary[];
  conflictActivity: ActivityLogEntry[];
  projectionHistory: { version: number; createdAt: string; reason: string }[];
  pagination: { nextCursors: Record<string, string | null> };
}
```

MVP vs later: MVP shows source, values, tier, timestamp, chain, linked conflicts, claim history. **Later:** interactive audit-hash *verification* (recompute + compare), cross-event provenance, archive (`include_archive=true`) browsing.

9. Timeline implementation

MVP: vertical chronological list of source-backed events from `GET /api/v1/public/events/{slug}/timeline` . Each `TimelineItem` shows time, summary, source(s), and a conflict/uncertainty indicator (amber dot when the underlying field is in `unresolved_conflict_fields`). Clicking an item drives the shared `EvidencePanel` . Filters: by source and date range.

Components: `TimelineList` , `TimelineItem` , `TimelineFilters` , reuse `EvidencePanel` , `StatusChip` .

Defer: Chronos *extraction* triggering (`POST /chronos/events/{id}/extract`) and sequence-review workflow (`/chronos/reviews/pending`) — those are curation actions, V1. MVP is read-only timeline.

10. PDF / docket ingestion module

Mirror the NTSB importer exactly — do not build a parallel ingestion path. The NTSB importer proves the pattern: a *pure mapping core* (no I/O), an *infrastructure reader* (the only format-touching code), a *CLI/route* entry, all feeding the existing `IngestSourceData` use case. Replicate that three-layer split.

Proposed files (note: package root is `atlas` , not `atlas_argus`):

```
src/atlas/application/ingestion/sources/document_extract.py    # pure: extracted text/fields → IngestionClaimDTO submission. No I/O.
src/atlas/infrastructure/ingestion/pdf_reader.py              # the only PDF-touching code: bytes → page text + metadata
src/atlas/infrastructure/storage/object_store.py              # S3-compatible put/get; tenant-scoped keys (new)
src/atlas/presentation/api/routers/documents.py              # upload + ingest route
src/atlas/presentation/api/schemas/documents.py              # request/response DTOs
src/atlas/presentation/cli/documents.py                      # optional bulk CLI (mirrors cli/ntsb.py)
tests/application/ingestion/test_document_extract.py           # unit tests on the pure mapper, no DB/no PDF lib
```

How it works (each piece reuses existing abstractions):

- **Uploaded file → Source.** On upload, resolve-or-create a `Source` (kind= `EXTERNAL` , an appropriate `reliability_tier`) exactly as the NTSB loader resolves its `Source` . Store the original file bytes in object storage; record metadata in a new `uploaded_documents` row (see §13).
- **Parsed text → claims.** `pdf_reader.py` extracts text + page map. `document_extract.py` (pure) turns extracted fields into `IngestionClaimDTO(field_name, field_value)` plus a `raw_payload` that **preserves the full raw text**. It then submits via the existing `POST /api/v1/ingestion/sources/{source_id}` / `IngestSourceData` — so conflict detection, projections, search, and causality work with **zero downstream change**.
- **Raw preservation.** Keep the original file in object storage *and* the full extracted text in `raw_payload` . This matches the codebase's “preserve raw value under canonical field name” philosophy and is essential for legal defensibility.
- **Avoid overconfident extraction (MVP strategy).** Do **not** require accurate AI field extraction. MVP = (a) store the file, (b) extract plain text, (c) create *at most* a few high-confidence claims from obvious structured fields (e.g. a docket/accession number, a clearly labeled date), and (d) leave everything else as raw text for **manual claim creation/review**. A wrong auto-extracted claim in a litigation tool is worse than no claim. Conservative-by-default mirrors the existing normalizer (identity-critical errors reject; non-identity failures keep raw).
- **Error handling.** Parsing failures never corrupt state: store the file + mark the document `parse_status=failed` , surface it in `IngestionStatus` UI, never fabricate claims. Encrypted/scanned PDFs → flag for OCR (later) rather than guessing.
- **Metadata stored:** filename, content hash (sha256), size, mime, uploader `user_id` , `source_id` , `event_id` (if known), page count, `parse_status` , object-store key, `created_at` .

MVP minimal parsing library: `pypdf` for text extraction (pure-Python, no native deps). Add OCR (`ocrmypdf` /Tesseract) only later for scanned dockets.

Upload/ingest route:

```
POST /api/v1/documents          # multipart upload → store file, create document row, return document_id
POST /api/v1/documents/{id}/ingest# parse + submit claims via IngestSourceData (reviewer/admin)
GET  /api/v1/documents?event_id= # list documents + parse/ingest status
GET  /api/v1/documents/{id}/text # raw extracted text for manual review
```

11. Report export module

Pure read + render — no new evidence data, reads the projection + provenance + conflict history + audit hashes.

Proposed files:

```
src/atlas/application/use_cases/export_case_report.py # gathers projection + provenance + history → ReportModel
src/atlas/domain/publication/report_model.py        # the report value object (sections, disclaimer)
src/atlas/infrastructure/reporting/html_renderer.py  # ReportModel → HTML (Jinja2 template)
src/atlas/infrastructure/reporting/pdf_renderer.py   # HTML → PDF (WeasyPrint)
src/atlas/infrastructure/reporting/templates/report.html
src/atlas/presentation/api/routers/reports.py
src/atlas/presentation/api/schemas/reports.py
tests/application/use_cases/test_export_case_report.py # asserts every fact carries a source ref
```

Reads from existing APIs/models: `QueryAccidentPublicView` (projection), `QueryProvenance` (claims/sources/history), `QueryConflictHistory` (resolution log), audit hashes from the audit router/data. No new reads invented.

HTML-first, then PDF. Render Jinja2 → HTML (also useful as an in-app preview), then HTML → PDF. **Library: WeasyPrint** — best CSS/print fidelity for document-style legal output (page numbers, headers/footers, footnotes) and pure-Python-friendly. (ReportLab is lower-level and slower to build legal layouts; wkhtmltopdf is effectively unmaintained.)

Court/professional friendliness: serif body, numbered sections, every fact footnoted with `source · ingestion timestamp · reliability tier`, a resolution-history appendix (who resolved what, when, with rationale), an audit-hash appendix for independent verification, page numbers + case caption in the running header, and a mandatory disclaimer page (“Atlas organizes and reconciles source evidence; it does not establish fault or liability; not legal advice”). Pull this disclaimer text from the blueprint §9.

Versioning + storage: each generate creates an immutable `report_exports` row (see §13) with a monotonically increasing `version` per case, the input projection version, a content hash, and the object-store key of the rendered PDF. Never overwrite — reports are evidence of what was believed at a point in time.

Routes + frontend flow:

```
GET  /api/v1/reports/{caseId}/preview      # render HTML preview (no persistence)
POST /api/v1/reports/{caseId}/generate     # render+store PDF, return report_id + version (reviewer/admin)
GET  /api/v1/reports/{reportId}/download   # authorized stream of stored PDF
GET  /api/v1/reports?event_id=             # list past exports for the case
```

Frontend: `ReportPreview` (renders the HTML preview inline) → `GenerateButton` (creates a versioned PDF) → `ReportList` (history with download). Download authorization checked at the BFF *and* enforced by role/tenant on FastAPI.

12. API mapping

Frontend screen	Needed data	Existing endpoint	Missing endpoint	Notes
Dashboard	metrics, recent	GET /api/v1/admin/metrics , GET /api/v1/public/events	—	admin-only metrics; gate for analyst
Case detail	projection	GET /api/v1/accidents/{id}	—	ProjectionFieldsDTO
Claims	claims + sources	GET /api/v1/accidents/{id}/provenance	(optional) GET /accidents/{id}/claims thin view	provenance already returns claims
Conflicts	conflict list	GET /api/v1/conflicts?event_id=	—	offset pagination

Resolve / reopen	mutate	POST /api/v1/conflicts/{id}/resolve / reopen	—	must handle 409 + expected_version
Conflict history	activity log	GET /api/v1/conflicts/{id}/history	—	keyset cursor
Provenance	full chain	GET /api/v1/accidents/{id}/provenance	—	sectioned cursors
Timeline	sequence	GET /api/v1/public/events/{slug}/timeline	—	needs slug; map event→slug
Sources	source list	GET /api/v1/hermes/sources	—	reuse for source registry
Upload	store + ingest	—	POST /api/v1/documents , POST /api/v1/documents/{id}/ingest	\$10
Reports	preview/generate/download	—	GET/POST /api/v1/reports/...	\$11
Audit	activity	GET /api/v1/audit/...	—	read-only
Team/settings	members/keys	GET /api/v1/admin , /api/v1/tenancy	BFF GET/POST /auth/users	human-user CRUD lives in BFF
Auth	login/me	—	BFF POST /auth/login , GET /auth/me , POST /auth/logout	not FastAPI
Leads	capture	—	BFF POST /api/leads	not FastAPI

Slug note: conflicts/accidents are keyed by `event_id` (UUID); public timeline/evidence by `slug`. The BFF should resolve and cache `event_id ↔ slug` (from `GET /public/events`) so the app can navigate by a single case identifier.

13. Database changes

These belong to the **BFF/identity layer**, not the Atlas core schema — keep them in a separate database/schema so you don’t entangle human-auth concerns with the append-only evidence store. Only `uploaded_documents` and `report_exports` touch Atlas-adjacent data.

Table / model	Purpose	Key fields	Needed now?
<code>users</code>	Human login identity	id, email (unique), password_hash, role, tenant_id, atlas_user_id (UUID stamped into history), encrypted_api_key, created_at, is_active	Yes (BFF)
<code>sessions</code>	Server-side sessions	id (opaque), user_id, created_at, expires_at, ip, user_agent	Yes (BFF) — or Redis
<code>request_access_leads</code>	Marketing lead capture	id, org_name, contact_email, role_type, use_case, message, created_at	Yes (BFF)
<code>uploaded_documents</code>	File metadata for PDF ingestion	id, source_id, event_id, filename, content_sha256, size, mime, page_count, parse_status, object_key, uploaded_by, created_at	Yes (Atlas-side, new migration)
<code>report_exports</code>	Versioned generated reports	id, event_id, version, projection_version, content_sha256, object_key, generated_by, created_at	Yes (Atlas-side, new migration)
Roles/permissions table	Fine-grained RBAC	—	No — reuse <code>ApiKeyModel.role</code> ; don’t duplicate
Human <code>user_id ↔</code> Atlas mapping	Audit attribution	(column on <code>users</code>)	Yes — store the same UUID you mint into the user’s API key so <code>ClaimHistory.modifier_id</code> is meaningful

Do **not** add new claim/conflict/projection structures — they exist and are append-only by design.

14. Security implementation checklist

Control	Status	Work
No raw API keys in frontend	✅ pattern enabled	Enforce via BFF; key encrypted server-side, never forwarded
Secure sessions	❌ new	BFF sessions (Redis), rotation on login, idle + absolute timeout
CSRF protection	❌ new	Double-submit token or <code>SameSite=Lax</code> + custom header check on BFF mutations
SameSite / HttpOnly / Secure / <code>__Host-</code> cookies	❌ new	Set on session cookie
File upload validation	❌ new	MIME + magic-byte sniff, extension allowlist (pdf), reject mismatches
File size limits	❌ new	Cap at BFF and FastAPI (e.g. 50 MB); stream, don't buffer whole
Malware scanning placeholder	❌ new	Hook (ClamAV/async queue) before marking a document ingestible
Tenant isolation	✅ strong (RLS, NOBYPASSRLS)	Keep tenant binding on per-user keys; never use a superuser key from BFF
RBAC	✅ exists	UI gating + server <code>require_role</code> (already enforced)
Audit logging	✅ exists (hash chains, ClaimHistory)	Ensure BFF passes the correct per-user key so <code>user_id</code> is attributed
Rate limiting	✅ configurable	Enable in prod (<code>config.py</code>); add BFF-level login throttle
CORS	✅ configurable	App + BFF same origin ideally; lock <code>cors_origins</code> , no * with credentials
HSTS	✅ configurable	Enable behind TLS (<code>hsts_enabled=true</code>)
Object-storage access controls	❌ new	Private bucket, tenant-scoped keys, time-limited signed URLs for download
Report download authorization	❌ new	Check role/tenant at BFF + FastAPI before streaming
Logging without leaking evidence	⚠ verify	Never log <code>raw_payload</code> , claim values, file contents, or API keys

15. Design system implementation

Legal-grade, serious, dense-but-calm. Implement as Tailwind tokens + shadcn theme.

- **Colors:** anchor deep navy/slate (`slate-800/900`); neutral grays for surfaces; **semantic-only** accents — `amber` = conflict/disputed, `green` = resolved, `red` = destructive/retracted, `gray` = superseded. No decorative color, no gradients.
- **Typography:** UI sans (Inter/IBM Plex Sans); **serif for report exports** (document feel). Generous line-height in evidence/claim text. Mono for hashes/IDs.
- **Spacing:** 8px grid; strong left-alignment; predictable 3-column scaffolding.
- **Tables:** thin rules, sticky header, zebra off (use rules), right-align numeric, sortable; dense row height — investigators want volume.
- **Status chips:** human-readable text ("Disputed", "Resolved by curator", "Superseded"), **not** numeric confidence bars. Any score shown is *evidence-support confidence*, never causal probability (carry the codebase's epistemic discipline into the UI).
- **Buttons:** solid primary (Resolve, Generate report), outline secondary; destructive (Reopen, Retract) visually distinct + confirmation dialog.
- **Forms:** inline validation, required-rationale fields prominent, mutually-exclusive choices as radios.
- **Modals/drawers:** right-side `Drawer` for evidence + history; `Dialog` for confirmations; never nest.
- **Empty states:** plain, honest ("No open conflicts for this case.").
- **Loading:** skeletons, never spinners-on-blank.
- **Error states:** inline, specific; the 409 conflict-modified banner is a designed state, not a toast.
- **Dashboard cards:** flat, bordered, label + number + small delta; no sparkline noise.

- **Evidence panels:** label/value pairs, monospace for hashes, copy buttons.
- **Report preview:** serif, paginated look, footnote markers, mirrors the PDF output.

Avoid: glassmorphism, animated hero blobs, emoji, “AI sparkle,” any invented/decorative metrics.

16. Development phases

Phase	Goal	Files / components	Backend work	Frontend work	Tests	Definition of done
1 — Repo + frontend foundation	Monorepo, app shell builds	apps/web scaffold, lib/api, types/, Tailwind/shadcn theme, query client	none	Vite+TS+Router+Query+Tailwind; design tokens	lint/typecheck/CI green	pnpm dev serves an empty authed-app shell hitting a stub BFF
2 — Public website	Lead capture live	apps/marketing pages + RequestAccessForm	BFF POST /api/leads, leads table	All marketing pages w/ blueprint copy	form e2e	site deploys; lead lands in DB; legal pages present
3 — Browser-safe auth + app shell	Login + navigate	BFF /auth/*, users / sessions, per-user key bridge; CaseSwitcher, AppLayout	mint per-user ApiKeyModel; session store	login, protected routes, case list via /public/events	auth + RBAC gating	user logs in, sees seeded Colgan 3407 , navigates; no key in browser
4 — Case dashboard + claims	See the record	CaseWorkspaceLayout, ClaimsTable, ClaimDetail, EvidencePanel v1	none (reads exist)	3-column layout; projection + provenance reads	render tests	claims + per-field source visible in 3 columns
5 — Conflict queue + provenance	Resolve on the record	ConflictTable/Detail, ResolveForm, ReopenForm, ConflictHistoryPanel, full EvidencePanel	none (resolve/reopen exist)	optimistic mutations; 409 handling ; keyboard shortcuts	mutation + 409 + permission tests	reviewer resolves a real conflict with rationale → ClaimHistory updated; 409 path verified
6 — Timeline	Reconstruct sequence	TimelineList/Item, TimelineFilters	none	timeline read; conflict indicators; evidence link	render tests	timeline shows source-backed, conflict-flagged events; click → evidence
7 — PDF/docket upload + ingestion	Real evidence in	documents.py router/schemas, pdf_reader.py, document_extract.py, object_store.py, uploaded_documents migration; DocumentUpload, IngestionStatus	new adapter + storage + routes	upload + status + manual review	unit (pure mapper) + integration (upload→ingest)	upload a PDF docket → file stored, raw text preserved, conservative claims created, conflicts surface
8 — Report preview/export	The paid deliverable	export_case_report.py, report_model.py, renderers + template, reports.py, report_exports migration; ReportPreview, ReportList	new export module	preview + generate + download	export content tests (every fact has a source)	generate a versioned PDF with projection + sources + resolution history + audit hashes + disclaimer

9 — Security + production readiness	Pass a security review	CSRF, cookie flags, upload validation, signed download URLs, rate-limit/HSTS on, monitoring	hardening	security UX states; audit read-only UI	security/e2e	security questionnaire answerable from real evidence; tenant isolation demonstrated
--	------------------------	---	-----------	--	--------------	---

17. First coding task

Update 2026-05-30: the BFF skeleton now exists in atlas-bff-docs-reports.zip. The first coding task is no longer to create a BFF from scratch; it is to wire, harden, and verify the existing skeleton against the current FastAPI/API-key backend.

Task name: Wire existing BFF skeleton + authenticated GET /api/cases passthrough.

Goal: A logged-in human, holding only an HttpOnly session cookie, can load the case list while the BFF, not the browser, calls FastAPI with a server-side API key. This locks in the browser-safe security boundary before feature work.

Files to inspect / complete:

- services/bff/src/server.ts - app bootstrap, cookie/session middleware, CORS to the web origin.
- services/bff/src/auth/routes.ts - POST /auth/login, POST /auth/logout, GET /auth/me.
- services/bff/src/auth/users.ts - minimal user store, password hash, encrypted Atlas API key handling.
- services/bff/src/atlasClient.ts - server-side client that injects X-API-Key and strips upstream secrets.
- services/bff/src/cases/routes.ts - GET /api/cases passthrough, camelCase shaping, safe JSON.
- services/bff/migrations/* - users and sessions, if not already applied in the target environment.
- apps/web/src/lib/api.ts and auth/case routes - point the browser only at the BFF origin.

Expected result:

Visiting /app/cases unauthenticated redirects to /login. After login, the case list renders from real /api/v1/public/events data fetched by the BFF. DevTools shows the session cookie is HttpOnly and no X-API-Key appears in any browser request.

Acceptance criteria:

- POST /auth/login with valid credentials sets a Secure, SameSite=Lax, preferably __Host- session cookie; invalid credentials return 401.
- GET /api/cases without a session returns 401 from the BFF before FastAPI is reached.
- GET /api/cases with a session returns camelCased case data sourced from /api/v1/public/events.
- The Atlas API key is never present in browser responses, JS bundles, request headers, or local storage.
- GET /auth/me returns email and role data suitable for analyst/reviewer/admin UI gating.
- POST /auth/logout invalidates the session; subsequent /api/cases returns 401.

Test plan:

- BFF unit: login success/failure, session issue/expiry, atlasClient attaches key and strips it from responses.
- BFF integration: /api/cases unauthenticated -> 401, authenticated -> 200 with shaped data.
- Web e2e: unauthenticated redirect -> login -> cases render; assert no x-api-key header and correct cookie flags.
- Security assertion: grep the web bundle for the seeded key value -> absent.

The first 10 development tasks, in order

1. Wire existing BFF skeleton + GET /api/cases passthrough - establishes and proves the security boundary.
2. Monorepo + design tokens - apps/web, apps/marketing, services/bff; Tailwind/shadcn theme, shared TS types, CI lint/typecheck/test.
3. App shell + case switcher + 3-column CaseWorkspaceLayout - sidebar, top bar, protected routing, empty workspace.
4. Demo seed script: real Colgan 3407 data - every screen/screenshot is honest from day one.
5. Claims view + EvidencePanel v1 - projection + provenance reads in the 3-column layout.
6. Conflict queue read path - ConflictTable + ConflictDetailPanel + history, over GET /conflicts, read-only first.
7. Resolve/reopen write path with 409 handling - optimistic mutations, expected_version, conflict-modified banner, required rationale, role gating.
8. Complete document upload + conservative ingestion - finish existing Documents.tsx route plus storage and parser workflow.
9. Complete report export module - finish existing Reports.tsx route plus WeasyPrint renderer, versioned report_exports, preview/generate/download.
10. Public marketing site + request-access - ship in parallel; lead capture to the BFF; legal pages to counsel; security/compliance docs early.

Net: tasks 1-3 are foundation; 4-7 prove the core product; 8-9 close document/report gaps now that the scaffold routes exist; 10 feeds the funnel. The backend remains the heavy lift already done; new code should wire and harden existing pieces before creating new scaffolds.